

Python_BT

0.1.0

Smartron India Pvt. Ltd.

Contents

1	README	1
2	Namespace Index	3
2.1	Namespace List	3
3	Hierarchical Index	5
3.1	Class Hierarchy	5
4	Class Index	7
4.1	Class List	7
5	Namespace Documentation	9
5.1	python_BT Namespace Reference	9
5.1.1	Detailed Description	9
5.2	python_BT.bluezutils Namespace Reference	9
5.2.1	Detailed Description	10
5.2.2	Function Documentation	10
5.2.2.1	find_adapter()	10
5.2.2.2	find_adapter_in_objects()	10
5.2.2.3	find_device()	10
5.2.2.4	find_device_in_objects()	11
5.2.2.5	find_media()	11
5.2.2.6	find_media_in_objects()	11
5.2.2.7	find_mediaplayer()	11
5.2.2.8	find_mediaplayer_in_objects()	11

5.3	python_BT.btlogger Namespace Reference	12
5.3.1	Detailed Description	12
5.3.2	Function Documentation	12
5.3.2.1	log_init()	12
5.4	python_BT.btmain Namespace Reference	12
5.4.1	Detailed Description	13
5.4.2	Function Documentation	13
5.4.2.1	bt_adapter_name()	13
5.4.2.2	bt_avrcp_command()	14
5.4.2.3	bt_discoverable()	14
5.4.2.4	bt_discovery_status()	14
5.4.2.5	bt_list_connected_dev()	14
5.4.2.6	bt_list_paired_dev()	15
5.4.2.7	bt_process()	15
5.4.2.8	bt_start_discovery()	15
5.4.2.9	bt_stop_discovery()	15
5.4.2.10	bt_switch()	16
5.4.2.11	check_adapter()	16
5.4.2.12	list_devices()	16
5.4.2.13	print_discovery()	16
5.4.2.14	scan_func()	17
5.4.2.15	set_adapter()	17
5.5	python_BT.config Namespace Reference	17
5.5.1	Detailed Description	17
6	Class Documentation	19
6.1	python_BT.bluezutils.BTError Class Reference	19
6.1.1	Detailed Description	19
6.2	python_BT.btmain.BTError Class Reference	19
6.2.1	Detailed Description	19

Chapter 1

README

Readme for Bt python libraries

1. bluezutils.py

BlueZ Utilities is the Library used to manage the BlueZ Objects and find the Objects path via D-Bus Interface.

BlueZ Utilities Functionalities-

- Get the Managed Objects of Bluez Object Manager
- Finds the availability of Adapter and Object Path
- Finds the availability of Device and Object Path
- Finds the availability of Media and Object Path on the Bluetooth Adapter
- Finds the Mediaplayer and Object path of the Bluetooth Adapter

2. btagent.py

Module is used to Access the API calls

Module Functionalities -

- Setting adapter properties for discoverable mode and adapter custom name
- checking the power of BT adapter
- checks for the adapter Discovering mode, starts and stops the discovering process
- Control BT Adapter Power
- Listing Connected, Paired and Discovered Devices
- Performs the AVRCP commands on Source Media Player

3. btlogger.py

Initialisation for BT Module Logger Configurations

BT Logger Functionality -

- Setting Logger level, Format and Logging Handler

4. init.py

library for initialization of API from btagent.py

5. config.py

Global configurations of D-bus Interfaces path for BT Module

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

python_BT	9
python_BT.bluezutils	9
python_BT.btlogger	12
python_BT.btmain	12
python_BT.config	17

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Exception	
python_BT.bluezutils.BTError	19
python_BT.btmain.BTError	19

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

python_BT.bluezutils.BTError	19
python_BT.btmain.BTError	19

Chapter 5

Namespace Documentation

5.1 python_BT Namespace Reference

Namespaces

- [bluezutils](#)
- [btlogger](#)
- [btmain](#)
- [config](#)

5.1.1 Detailed Description

Initialisation of Python_BT Module API calls

5.2 python_BT.bluezutils Namespace Reference

Classes

- class [BTErrors](#)

Functions

- def [get_managed_objects](#) ()
Getting D-Bus Managed Objects from the Bluez Object Manager.
- def [find_adapter](#) (pattern=None)
- def [find_adapter_in_objects](#) (objects, pattern=None)
- def [find_device](#) (device_address, adapter_pattern=None)
- def [find_device_in_objects](#) (objects, device_address, adapter_pattern=None)
- def [find_media](#) (pattern=None)
- def [find_media_in_objects](#) (objects, pattern=None)
- def [find_mediaplayer](#) (device_address, adapter_pattern=None)
- def [find_mediaplayer_in_objects](#) (objects, device_address, adapter_pattern=None)

Variables

- `log = btlogger.log_init()`
initialising the logger from btlogger

5.2.1 Detailed Description

Bluetooth_Manager: Supporting for A2DP & AVRCP profiles

5.2.2 Function Documentation

5.2.2.1 find_adapter()

```
def python_BT.bluezutils.find_adapter (
    pattern = None )
```

Function returns the adapter object paths from the Object Manager

5.2.2.2 find_adapter_in_objects()

```
def python_BT.bluezutils.find_adapter_in_objects (
    objects,
    pattern = None )
```

Function takes the Managed Objects, check for the Adapter Interfaces and return the Object path

5.2.2.3 find_device()

```
def python_BT.bluezutils.find_device (
    device_address,
    adapter_pattern = None )
```

Finding return the device object paths from the Adapter Interface in Object Manager

5.2.2.4 find_device_in_objects()

```
def python_BT.bluezutils.find_device_in_objects (
    objects,
    device_address,
    adapter_pattern = None )
```

Function takes the Managed Objects, check for the Device Interfaces and return the Device Object path

5.2.2.5 find_media()

```
def python_BT.bluezutils.find_media (
    pattern = None )
```

Function returns the Media object paths from the Object Manager

5.2.2.6 find_media_in_objects()

```
def python_BT.bluezutils.find_media_in_objects (
    objects,
    pattern = None )
```

Function takes the Managed Objects, check for the Media Interfaces and return the Object path

5.2.2.7 find_mediaplayer()

```
def python_BT.bluezutils.find_mediaplayer (
    device_address,
    adapter_pattern = None )
```

Finding return the device object paths from the Adapter Interface in Object Manager

5.2.2.8 find_mediaplayer_in_objects()

```
def python_BT.bluezutils.find_mediaplayer_in_objects (
    objects,
    device_address,
    adapter_pattern = None )
```

Function takes the Managed Objects, check for the Adapter Interfaces and return the Mediaplayer Object path

5.3 python_BT.btlogger Namespace Reference

Functions

- def [log_init](#) ()
Function call for Logger Initialisation.

5.3.1 Detailed Description

BT Logger Configuration

5.3.2 Function Documentation

5.3.2.1 log_init()

```
def python_BT.btlogger.log_init ( )
```

Function call for Logger Initialisation.

- Initialising the Logger
- Setting the Logging Level
Possible Logging Levels = logging.DEBUG, logging.INFO, logging.ERROR, logging.NOTSET
- Setting Logger Format

5.4 python_BT.btmain Namespace Reference

Classes

- class [BTError](#)

Functions

- def [set_adapter](#) (property_keyword, value_keyword)
BT module Configuration.
- def [check_adapter](#) (check_keyword)
Shifu-BT validating the status of power and Scan mode: check_keyword : String, possible values= ["power", "scan"].
- def [bt_switch](#) (switch_cmd)
BLUETOOTH MODULE POWER API CALL: switch_cmd : Boolean, possible values= [True, False].
- def [scan_func](#) (cmd)
BT SCAN PROCESSING FUNCTION: cmd : String, possible values= ["on", "off"].
- def [bt_stop_discovery](#) ()
API CALL TO STOP the SCAN process.
- def [list_devices](#) (list_keyword)

LIST OF ALL DEVICES

list_keyword: String type, Possible values: ["Paired", "Connected", "RSSI", "RSSIprev"].

- def `print_discovery` ()
API CALL FOR SCANNED DEVICE LIST.
- def `bt_start_discovery` (DISCOVERY_TIME=30)
API Call FOR START SCANNING PROCESS.
- def `bt_discovery_status` ()
API CALL FOR SCANNING MODE STATUS.
- def `bt_discoverable` (discoverable_time)
API CALL FOR DISCOVERABLE MODE : *discoverable_time = uint32_t (in secs)*
- def `bt_adapter_name` (name_utf8)
API CALL TO MODIFY Shifu-BT NAME : *name_utf8: utf-8 / String.*
- def `bt_process` (mac_addr, process_cmd)
BLUETOOTH ACTION API CALL : *mac_addr : String type, format= "XX:XX:XX:XX:XX:XX" process_cmd : String type, possible values: ["unpair", "connect", "disconnect"].*
- def `bt_list_paired_dev` ()
API CALL FOR PAIRED DEVICES LIST.
- def `bt_list_connected_dev` ()
API CALL FOR CONNECTED DEVICE NAME & MAC ADDR.
- def `bt_avrcp_command` (mac_addr, player_cmd)
AVRCP API CALL: *mac_addr : String, format= "XX:XX:XX:XX:XX:XX" process_cmd : String, possible values: ["play", "pause", "stop", "next", "previous", "restart"].*

Variables

- dictionary `discovered_bt` = {}
To store the list of external BT scanned devices.
- `log` = `btlogger.log_init()`
To initialize the logger using btlogger module.

5.4.1 Detailed Description

Bluetooth_Manager: Supporting Shifu-thub with A2DP & AVRCP profiles

5.4.2 Function Documentation

5.4.2.1 `bt_adapter_name()`

```
def python_BT.btmain.bt_adapter_name (
    name_utf8 )
```

API CALL TO MODIFY Shifu-BT NAME : *name_utf8: utf-8 / String.*

API call to modify the Shifu-BT name. utf-8 format data will be converted to Unicode

5.4.2.2 bt_avrcp_command()

```
def python_BT.btmain.bt_avrcp_command (
    mac_addr,
    player_cmd )
```

AVRCP API CALL: mac_addr : String, format= "XX:XX:XX:XX:XX:XX" process_cmd : String, possible values: ["play", "pause", "stop", "next", "previous", "restart"].

API call will be used for AVRCP commands: play, pause, stop, next, previous and restart the track

5.4.2.3 bt_discoverable()

```
def python_BT.btmain.bt_discoverable (
    discoverable_time )
```

API CALL FOR DISCOVERABLE MODE : discoverable_time = uint32_t (in secs)

API call to keep Shifu-BT in discoverable mode based on the requested time in secs.
If discoverable_time is set to 0, Shifu-BT will be always in discoverable mode to other devices.

5.4.2.4 bt_discovery_status()

```
def python_BT.btmain.bt_discovery_status ( )
```

API CALL FOR SCANNING MODE STATUS.

API to know the BT scanning process

5.4.2.5 bt_list_connected_dev()

```
def python_BT.btmain.bt_list_connected_dev ( )
```

API CALL FOR CONNECTED DEVICE NAME & MAC ADDR.

API for Connected BT Devices list

5.4.2.6 bt_list_paired_dev()

```
def python_BT.btmain.bt_list_paired_dev ( )
```

API CALL FOR PAIRED DEVICES LIST.

API for Paired BT Devices list

5.4.2.7 bt_process()

```
def python_BT.btmain.bt_process (
    mac_addr,
    process_cmd )
```

BLUETOOTH ACTION API CALL : mac_addr : String type, format= "XX:XX:XX:XX:XX:XX" process_cmd : String type, possible values: ["unpair", "connect", "disconnect"].

Function is responsible to handle pairing & connection, disconnection and unpairing the external BT device

5.4.2.8 bt_start_discovery()

```
def python_BT.btmain.bt_start_discovery (
    DISCOVERY_TIME = 30 )
```

API Call FOR START SCANNING PROCESS.

API call to initialise the BT scan process for the defined period of time

5.4.2.9 bt_stop_discovery()

```
def python_BT.btmain.bt_stop_discovery ( )
```

API CALL TO STOP the SCAN process.

API call to stop the scan process

5.4.2.10 `bt_switch()`

```
def python_BT.btmain.bt_switch (
    switch_cmd )
```

BLUETOOTH MODULE POWER API CALL: switch_cmd : Boolean, possible values= [True, False].

Used to switch bluetooth power ON & OFF

5.4.2.11 `check_adapter()`

```
def python_BT.btmain.check_adapter (
    check_keyword )
```

Shifu-BT validating the status of power and Scan mode: check_keyword : String, possible values= ["power", "scan"].

Verifies the BT Adapter power status before starting any operations
and also verifies BT scanning status

5.4.2.12 `list_devices()`

```
def python_BT.btmain.list_devices (
    list_keyword )
```

LIST OF ALL DEVICES

list_keyword: String type, Possible values: ["Paired", "Connected", "RSSI", "RSSIprev"].

Provides the list of Scanned, Paired and Connected device
and also used to clear the list of previously scanned devices

5.4.2.13 `print_discovery()`

```
def python_BT.btmain.print_discovery ( )
```

API CALL FOR SCANNED DEVICE LIST.

API call to list all the external scanned BT devices

5.4.2.14 scan_func()

```
def python_BT.btmain.scan_func (
    cmd )
```

BT SCAN PROCESSING FUNCTION: cmd : String, possible values= ["on", "off"].

Responsible to start & Stop the BT scan process

5.4.2.15 set_adapter()

```
def python_BT.btmain.set_adapter (
    property_keyword,
    value_keyword )
```

BT module Configuration.

To configure Shifu-BT name and discoverable mode (To perform Shifu as visible to other BT devices). Once after the time-out, Shifu-BT won't be visible to external BT devices.

property_keyword: String, Possible values: ["discoverable", "name"]

Setting discoverable time: Property_keyword = "discoverable", value_keyword : int

Setting Name of adapter: Property Keyword = "name", value_keyword : utf-8

Used to configure Shifu-BT name and the Discoverable period.

5.5 python_BT.config Namespace Reference

Variables

- string `SERVICE_NAME` = "org.bluez"
Bluez Service name.
- string `ADAPTER_INTERFACE` = `SERVICE_NAME` + ".Adapter1"
Adapter Interface path.
- string `DEVICE_INTERFACE` = `SERVICE_NAME` + ".Device1"
Device Interface path.
- string `MEDIA_INTERFACE` = `SERVICE_NAME` + ".Media1"
Media Interface path.
- string `MEDIAPLAYER_INTERFACE` = `SERVICE_NAME` + ".MediaPlayer1"
Media Player Interface path.

5.5.1 Detailed Description

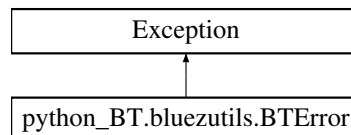
Configurational path variables for D-Bus Interface

Chapter 6

Class Documentation

6.1 python_BT.bluezutils.BTError Class Reference

Inheritance diagram for python_BT.bluezutils.BTError:



6.1.1 Detailed Description

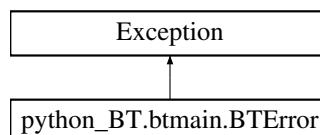
Base class for BT_Exceptions

The documentation for this class was generated from the following file:

- bluezutils.py

6.2 python_BT.btmain.BTError Class Reference

Inheritance diagram for python_BT.btmain.BTError:



6.2.1 Detailed Description

Base class for BT_Exceptions

The documentation for this class was generated from the following file:

- btmain.py

